

1)

O que cada algoritmo faz?

Algoritmo 1: Calcula a soma total de todos os elementos numéricos contidos em um vetor.

Algoritmo 2: Realiza uma busca linear para encontrar o índice de um valor específico dentro de um vetor, retornando -1 se não encontrado.

Algoritmo 3: Percorre o vetor para identificar e retornar o maior valor (máximo) entre todos os elementos.

Algoritmo 4: Compara os elementos do vetor com um valor de referência e conta quantos deles são estritamente maiores que esse limite.

2)

Editor

Simulator

Run

Step

Prev

Reset

Dump

Machine Code	Basic Code	Original Code
0x10000517	auipc x10 65536	la a0, vetor
0x00050513	addi x10 x10 0	la a0, vetor
0x10000597	auipc x11 65536	lw a1, n
0x00c5a583	lw x11 12(x11)	lw a1, n
0x008000ef	jal x1 8	jal ra, sum_vec
0x00a00893	addi x17 x0 10	li a7, 10
0x000002b3	add x5 x0 x0	add t0, zero, zero # t0 = soma = 0
0x00000333	add x6 x0 x0	add t1, zero, zero # t1 = i = 0
0x00b35e63	bge x6 x11 28	bge t1, a1, sum_end # se i >= n, encerra
0x00231393	slli x7 x6 2	slli t2, t1, 2 # t2 = i * 4 (ajuste de endereço para word)
0x007503b3	add x7 x10 x7	add t2, a0, t2 # t2 = &v[i]
0x0003ae03	lw x28 0(x7)	lw t3, 0(t2) # t3 = v[i]
0x01c282b3	add x5 x5 x28	add t0, t0, t3 # soma += v[i]

console output

Registers

Memory

zero	0x00000000
ra (x1)	0x00000000
sp (x2)	0x7fffffff0
gp (x3)	0x10000000
tp (x4)	0x00000000
t0 (x5)	0x00000000
t1 (x6)	0x00000000
t2 (x7)	0x00000000
s0 (x8)	0x00000000
s1 (x9)	0x00000000
a0 (x10)	0x00000000
a1 (x11)	0x00000000
a2 (x12)	0x00000000
a3 (x13)	0x00000000
a4 (x14)	0x00000000
a5 (x15)	0x00000000
a6 (x16)	0x00000000

Display Settings

Hex

Editor

Simulator

Run

Step

Prev

Reset

Dump

Machine Code	Basic Code	Original Code
0x10000517	auipc x10 65536	la a0, vetor
0x00050513	addi x10 x10 0	la a0, vetor
0x10000597	auipc x11 65536	lw a1, n
0x00c5a583	lw x11 12(x11)	lw a1, n
0x008000ef	jal x1 8	jal ra, sum_vec
0x00a00893	addi x17 x0 10	li a7, 10
0x000002b3	add x5 x0 x0	add t0, zero, zero # t0 = soma = 0
0x00000333	add x6 x0 x0	add t1, zero, zero # t1 = i = 0
0x00b35e63	bge x6 x11 28	bge t1, a1, sum_end # se i >= n, encerra
0x00231393	slli x7 x6 2	slli t2, t1, 2 # t2 = i * 4 (ajuste de endereço para word)
0x007503b3	add x7 x10 x7	add t2, a0, t2 # t2 = &v[i]
0x0003ae03	lw x28 0(x7)	lw t3, 0(t2) # t3 = v[i]
0x01c282b3	add x5 x5 x28	add t0, t0, t3 # soma += v[i]

console output

Registers

Memory

zero	0x00000000
ra (x1)	0x00000000
sp (x2)	0x7fffffff0
gp (x3)	0x10000000
tp (x4)	0x00000000
t0 (x5)	0x00000000
t1 (x6)	0x00000000
t2 (x7)	0x00000000
s0 (x8)	0x00000000
s1 (x9)	0x00000000
a0 (x10)	0x10000000
a1 (x11)	0x00000000
a2 (x12)	0x00000000
a3 (x13)	0x00000000
a4 (x14)	0x00000000
a5 (x15)	0x00000000
a6 (x16)	0x00000000

Display Settings

Hex

Editor

Simulator

Run

Step

Prev

Reset

Dump

Machine Code	Basic Code	Original Code
0x10000517	auipc x10 65536	la a0, vetor
0x00050513	addi x10 x10 0	la a0, vetor
0x10000597	auipc x11 65536	lw a1, n
0x00c5a583	lw x11 12(x11)	lw a1, n
0x008000ef	jal x1 8	jal ra, sum_vec
0x00a00893	addi x17 x0 10	li a7, 10
0x000002b3	add x5 x0 x0	add t0, zero, zero # t0 = soma = 0
0x00000333	add x6 x0 x0	add t1, zero, zero # t1 = i = 0
0x00b35e63	bge x6 x11 28	bge t1, a1, sum_end # se i >= n, encerra
0x00231393	slli x7 x6 2	slli t2, t1, 2 # t2 = i * 4 (ajuste de endereço para word)
0x007503b3	add x7 x10 x7	add t2, a0, t2 # t2 = &v[i]
0x0003ae03	lw x28 0(x7)	lw t3, 0(t2) # t3 = v[i]
0x01c282b3	add x5 x5 x28	add t0, t0, t3 # soma += v[i]

console output

Registers

Memory

zero	0x00000000
ra (x1)	0x00000000
sp (x2)	0x7fffffff
gp (x3)	0x10000000
tp (x4)	0x00000000
t0 (x5)	0x00000000
t1 (x6)	0x00000000
t2 (x7)	0x00000000
s0 (x8)	0x00000000
s1 (x9)	0x00000000
a0 (x10)	0x10000000
a1 (x11)	0x00000000
a2 (x12)	0x00000000
a3 (x13)	0x00000000
a4 (x14)	0x00000000
a5 (x15)	0x00000000
a6 (x16)	0x00000000

Display Settings

Hex

Editor

Simulator

Run

Step

Prev

Reset

Dump

Machine Code	Basic Code	Original Code
0x10000517	auipc x10 65536	la a0, vetor
0x00050513	addi x10 x10 0	la a0, vetor
0x10000597	auipc x11 65536	lw a1, n
0x00c5a583	lw x11 12(x11)	lw a1, n
0x008000ef	jal x1 8	jal ra, sum_vec
0x00a00893	addi x17 x0 10	li a7, 10
0x000002b3	add x5 x0 x0	add t0, zero, zero # t0 = soma = 0
0x00000333	add x6 x0 x0	add t1, zero, zero # t1 = i = 0
0x00b35e63	bge x6 x11 28	bge t1, a1, sum_end # se i >= n, encerra
0x00231393	slli x7 x6 2	slli t2, t1, 2 # t2 = i * 4 (ajuste de endereço para word)
0x007503b3	add x7 x10 x7	add t2, a0, t2 # t2 = &v[i]
0x0003ae03	lw x28 0(x7)	lw t3, 0(t2) # t3 = v[i]
0x01c282b3	add x5 x5 x28	add t0, t0, t3 # soma += v[i]

console output

Registers

Memory

zero	0x00000000
ra (x1)	0x00000000
sp (x2)	0x7fffffff
gp (x3)	0x10000000
tp (x4)	0x00000000
t0 (x5)	0x00000000
t1 (x6)	0x00000000
t2 (x7)	0x00000000
s0 (x8)	0x00000000
s1 (x9)	0x00000000
a0 (x10)	0x10000000
a1 (x11)	0x00000000
a2 (x12)	0x00000000
a3 (x13)	0x00000000
a4 (x14)	0x00000000
a5 (x15)	0x00000000
a6 (x16)	0x00000000

Display Settings

Hex

Print 1: Configuração Inicial (Main) O que está acontecendo: O programa prepara os dados antes de somar. Instrução la a0, vetor: Carrega o endereço base do array [5, 4, 3, 2, 1] no registrador a0. Instrução lw a1, n: Carrega o valor 5 (tamanho do vetor) da memória para o registrador a1. Instrução jal ra, sum_vec: Salta para a função de soma e salva o endereço de retorno em ra. Print 2: Inicialização da Função (sum_vec) O que está acontecendo: Preparando os contadores internos da função. Instrução add t0, zero, zero: Inicializa a variável de soma (s = 0) colocando o valor zero no registrador t0. Instrução add t1, zero, zero: Inicializa o índice do loop (i = 0) no registrador t1. Print 3: Dentro do Loop (sum_loop) O que está acontecendo: Onde a mágica acontece — lendo os valores e somando. Instrução bge t1, a1, sum_end: Verifica se o índice i já chegou no tamanho n. Se sim, sai do loop. Instrução slli t2,

t1, 2: Multiplica o índice por 4 para achar o deslocamento em bytes (cada número ocupa 4 bytes). Instrução lw t3, 0(t2): Busca o valor real do vetor na memória e joga no registrador t3. Instrução add t0, t0, t3: Acumula o valor lido na soma total (soma += v[i]). Print 4: Finalização e Retorno O que está acontecendo: Preparando o resultado final para o usuário. Instrução addi t1, t1, 1: Incrementa o índice (i++) para passar para o próximo número. Instrução j sum_loop: Salta de volta para o início do loop para processar o próximo item. Instrução li a7, 10: Prepara o sistema para encerrar o programa corretamente.

3) Analogia: Assembly RISC-V vs. Linguagens de Alto Nível (C/Python)

1. Instrução: add t0, t0, t3,

C/Python: soma = soma + valor

Parâmetros: t0 (registrador de destino) recebe a soma dos valores contidos em t0 (acumulador) e t3 (valor atual do vetor).

2. Instrução: addi t1, t1, 1,

C/Python: i++ ou i += 1

Parâmetros: Soma a constante imediata 1 ao valor do registrador t1 e armazena o resultado no próprio t1.

3. Instrução: lw t3, 0(t2),

C/Python: valor = vetor[i]

Parâmetros: Carrega no registrador t3 o dado vindo da memória, usando o endereço base em t2 com um deslocamento (offset) de 0 bytes.

4. Instrução: li a2, 3,

C/Python: busca = 3

Parâmetros: Carrega um número inteiro imediato (3) diretamente no registrador de argumento a2.

5. Instrução: bge t1, a1, sum_end,

C/Python: if (i >= n) break;

Parâmetros: Compara t1 (índice) com a1 (tamanho); se t1 for Maior ou Igual, o programa pula para o rótulo sum_end.

6. Instrução: beq t2, a2, ls_found,

C/Python: if (v[i] == x) return i;

Parâmetros: Compara t2 com a2; se forem iguais, a execução salta para o rótulo de sucesso ls_found.

7. Instrução: slli t2, t1, 2,

C/Python: índice * 4

Parâmetros: Realiza um deslocamento lógico para a esquerda em 2 bits no valor de t1, o que equivale a multiplicar por 4 para calcular o endereço de memória.

8. Instrução: la a0, vetor,

C/Python: ptr = &vetor

Parâmetros: Carrega o endereço de memória (ponteiro) onde o rótulo vetor começa para dentro do registrador a0.

9. Instrução: mv a0, t0,

C/Python: return soma

Parâmetros: Move (copia) o conteúdo do registrador temporário t0 para o registrador de retorno a0.

10. Instrução: j sum_loop,

C/Python: continue ou while(true)

Parâmetros: Realiza um salto incondicional direto para o rótulo sum_loop para repetir a execução do bloco de código.